

Found a bug? [Edit this page on GitHub](#).

SOSS

This chapter describes the system-of-systems synthesizer (SOSS), a tool which provides protocol translation between different subsystems. Such composite systems can be called The ROS-SOSS. To see the current implementation status, see the [SOSS repository](#).

Motivation and Introduction

The ecosystem of different message passing systems is vast and diverse. Without any one single system that is unanimously considered the best for all applications, we are left to consider how we can tie together disparate message passing systems to bridge the gap between the different kinds of applications that all play a critical role in a modern, intelligent robotics solution. The best protocols to use for inter-robot communication might not be the best for remote operator communication or for end-user communication.

This creates a scalability problem. If there are N different message passing frameworks being used in a robot deployment with M different message types being passed between them, then manually creating inter-compatibility bridges between them could become an $O(MN^2)$ complexity problem. This motivates us to use a highly modular, user-friendly integration system that allows the interoperability to be as automatic as possible for as many different message-passing frameworks as we can tie together. The $O(MN^2)$ problem can be reduced to $O(N)$ complexity, where a plugin is written for each N framework, and all M message types are automatically converted between their N different representations.

The integration service we use for this is called System-of-Systems Synthesizer, or SOSS. The base SOSS package is simply some abstract interfaces defined in a C++ library, along with a single `soass` application. Each different message-passing system will have its own plugin library, e.g. DDS-SOSS, Websocket-SOSS, ROS-SOSS, that implements the abstract interfaces of the base SOSS. When you run the `soass` application, you provide a single configuration file that describes how you want your different message-passing systems to bridge with each other. The `soass` application can then find plugins that meet the requirements of your configuration file and load those plugins as it starts up. When messages start to move within each message-passing system, the `soass` application will grab, translate, and push the messages across the different system boundaries according to the configuration file that was given to it. Any number of `soass` instances can be run at once, but they will run independently of each other, so it is important to make sure that their configurations do not overlap.

The base SOSS package also provides some CMake tools to help with auto-generating message definitions for message-passing systems that require static message definitions at compile-

time. For message-passing systems with dynamic message types, the plugin can automatically take care of the translation at runtime, so auto-generation is unnecessary.

For a deeper look at how to use SOSS, we recommend reading [the documentation provided by eProsima](#).

Found a bug? [Edit this page on GitHub](#).

Integration

This chapter describes the requirements and basic steps to integrate hardware with RMF. These include [mobile robots](#), [doors](#), [elevators](#) and [workcells](#). In each section, we will go through how to build and use the necessary ROS 2 packages and interfaces, as well as possible scenarios where such interactions occur.

RMF uses ROS 2 messages and topic interfaces to communicate between different components in the overall RMF system. In most cases we use components called Adapters to bridge between the hardware-specific interfaces and the general purpose interfaces of RMF. This chapter will discuss how to develop an RMF Adapter for different types of hardware components.

Found a bug? [Edit this page on GitHub](#).

Route Map Data Requirements for Integration with RMF

Motivation

RMF uses robot route maps to predict the navigation paths of robots working in the environment. RMF generates path predictions for all active robots in the environment which can be used to proactively avoid conflicts between the various robot path plans. This is often referred to as "traffic management" in RMF. Along with the traffic management, RMF can help enable multi-fleet visualization to building/robot operations staff, improve scheduling of resources (such as lifts and corridors), reduce robot deadlock and more.

Robot route maps in large buildings are complex and may evolve over time in response to customer requests and building renovations. As a result, RMF works best when scripts can automatically import robot route maps, and re-import them in the future after changes are made.

Minimum Map Information Required

- list of waypoints or nodes
 - name of waypoint
 - level name (B1, L1, L2, etc.)
 - (x, y) location in meters within the level
 - any special properties or flags, such as:
 - is this a dropoff/pickup parking point?
 - is this a charger?
 - is this a safe parking spot during an emergency alarm?
- list of edges or "travel lanes" between nodes
 - (start, end) waypoint names
 - two-way or one-way traffic?
 - if one-way, identify direction of travel
 - any other information, such as speed limit along this segment

Format Requirements

We can write import scripts to handle virtually any "open" file format that contains the required information. This includes, in order of preference:

- YAML
- XML
- plain text (space or comma-separated ASCII, etc.)
- DXF
- DWG
- SVG

Note that if the map data is provided in textual form, screenshots are helpful for "sanity-checking" the coordinate system and alignment with building features.

Traffic Editor

If the robot route map does not exist yet, then the [traffic editor tool](#) can be used to help create one. The traffic editor tool will also export the route map in an RMF-friendly format.

Found a bug? [Edit this page on GitHub](#).

Mobile Robot Fleet Integration

Here we will cover integrating a mobile robot fleet that offers the **Full Control** category of fleet adapter, as **discussed** in the [RMF Core Overview](#) chapter. This means we assume the mobile robot fleet manager allows us to specify explicit paths for the robot to follow, and that the path can be interrupted at any time and replaced with a new path. Furthermore, each robot's position will be updated live as the robots are moving.

Route Map

Before such a fleet can be integrated, you will need to procure or produce a route map as described in the [previous section](#). The fleet adapter uses the route map to plan out feasible routes for the vehicles under its control, taking into account the schedules of all other vehicles. It will also use the route map to decide out how to negotiate with other fleet adapters when a scheduling conflict arises. The adapter will only consider moving the robots along routes that are specified on the route map, so it is important that the route coverage is comprehensive. At the same time, if there are extraneous waypoints on the route map, the adapter might spend more time considering all the possibilities than what should really be needed, so it is a good idea to have a balance of comprehensiveness and leanness.

C++ API

The C++ API for **Full Control** automated guided vehicle (AGV) fleets can be found in the [rmf_fleet_adapter](#) package of the [rmf_ros2](#) repo. The API consists of four critical classes:

- [Adapter](#) - Initializes and maintains communication with the other core RMF systems. Use this to register one or more fleets and receive a `FleetUpdateHandle` for each fleet.
- [FleetUpdateHandle](#) - Allows you to configure a fleet by adding robots and specifying settings for the fleet (e.g. specifying what types of deliveries the fleet can perform). New robots can be added to the fleet at any time.
- [RobotUpdateHandle](#) - Use this to update the position of a robot and to notify the adapter if the robot's progress gets interrupted.
- [RobotCommandHandle](#) - This is a pure abstract interface class. The functions of this class must be implemented to call upon the API of the specific fleet manager that is being adapted.

The C++ API for **Easy Full Control** fleets provides a simple and more accessible way for users to integrate with the Full Control library without having to modify its internal logic. It can be found in the `rmf_fleet_adapter` package of the `rmf_ros2` repo. The `EasyFullControl` class contains helpful methods for users to create a `Configuration` object from YAML files encapsulating important fleet configuration parameters and navigation graphs, as well as to make their own fleet adapter with the `Configuration` object. The `add_robot(~)` method is provided for users to add robots to the new fleet adapter. This method takes in various callbacks that should be written by the user, and will be triggered whenever RMF is retrieving robot state information from the fleet or sending out commands to perform a particular process (navigation, docking, action, etc.). An example of the EasyFullControl fleet adapter can be found in `fleet_adapter.py` under the `rmf_demos` repo.

The C++ API for **Traffic Light Control** fleets (i.e. fleets that only allow RMF to pause/resume each mobile robot) can also be found in the `rmf_fleet_adapter` package of the `rmf_ros2` repo. The API reuses the `Adapter` class and requires users to initialize their fleet using either of the APIs [here](#). The user has the option to integrate via the `TrafficLight` API or for greater convenience, via the `EasyTrafficLight` API.

The basic workflow of developing a fleet adapter is the following:

1. Create an application that links to the `rmf_fleet_adapter` library.
2. Have the application read in runtime parameters in whatever way is desired (e.g. command line arguments, configuration file, ROS parameters, REST API calls, environment variables, etc).
3. Construct a route graph for each fleet that this application is providing the adapter for (a single adapter application can service any number of fleets), and/or parse the route graph from a YAML file using the `rmf_fleet_adapter::agv::parse_graph` utility.
4. Instantiate an `rmf_fleet_adapter::agv::Adapter` using `Adapter::make(~)` or `Adapter::init_and_make(~)`.
5. Add the fleets that the application will be responsible for adapting, and save the `rmf_fleet_adapter::agv::FleetUpdateHandlePtr` instances that are passed back.
6. Implement the `RobotCommandHandle` class for the fleet manager API that is being adapted.
7. Add the robots that the adapter is responsible for controlling. The robots can be added based on the startup configuration, or they can be dynamically added during runtime as they are discovered over the fleet manager API (or both).
 - When adding a robot, you will need to create a new instance of the custom `RobotCommandHandle` that you implemented.
 - You will also need to provide a callback that will be triggered when the adapter is finished registering the robot. This callback will provide you with a new `RobotUpdateHandle` for your robot. It is imperative to save this update handle so you can use it to update the robot's position over time.

8. As new information arrives from the fleet manager API, use the collection of `RobotUpdateHandle` classes to keep the adapter up-to-date on the robots' positions.

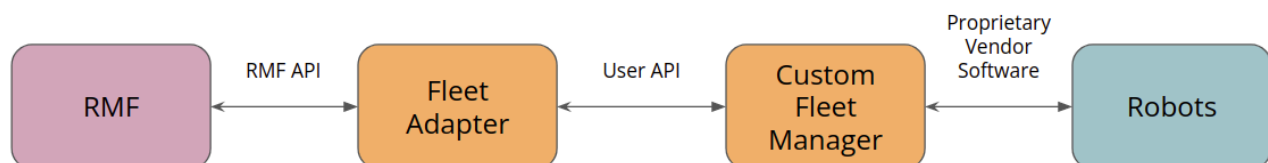
An example of a functioning fleet adapter application can be found in the [full_control backwards-compatibility adapter](#). This is a fleet adapter whose fleet-side API is the "Fleet Driver API", which is a deprecated prototype API for the RMF **Full Control** category of fleet adapters. This fleet adapter exists temporarily to maintain backwards compatibility with the old "Fleet Driver" implementations and to serve as an example of how to implement a fleet adapter using the new C++ API.

Python Bindings

You may also choose to use Python to implement your fleet adapter. You can find Python bindings for the C++ API in the [rmf_fleet_adapter_python repo](#). The Python bindings literally just port the C++ API into Python so that you can develop your fleet adapter using Python instead of C++. The above API and workflow are exactly the same, just in Python instead. This should be very useful for fleets that use REST APIs, because you'll have access to tools like [Swagger](#) which can help you generate client code for the fleet's REST API server.

Fleet Adapter Template

To make the process of integrating a robotic fleet with RMF even simpler, we have open-sourced a **Full Control template package** where users only need to update certain blocks of code with the API calls to their specific robot/fleet manager. This template uses the `EasyFullControl` API, where user-implemented robot callbacks are executed by RMF. This way, users can integrate RMF while using their preferred APIs between the fleet adapter and their robots. Do note that this template is just one of many ways to integrate fleets with REST or websocket based APIs. The following diagram illustrates how RMF can communicate with fleet robots using the APIs chosen by the user and the robot vendor.



This fleet adapter system is also integrated in our demos worlds with simulation robots, which is further elaborated in the next section.

You can follow these steps to build your fleet adapter on top of the given template:

1. Fill in the missing code in `RobotClientAPI.py`. With your chosen API (it could be ROS messages or any custom API), RMF will be able to command your robots and retrieve state updates. The crucial callbacks are:
 - `navigate()` : Commands your robot to a target location given the coordinates `[x, y, yaw]`.
 - `stop()` : Commands your robot to stop in place.
 - `execute_action()` : Triggers any custom action you would like your robot to perform when provided with a `category` and `description`, which detail the action name and task-specific parameters respectively. You may optionally want to make use of `RobotAPI.start_activity` to trigger different types of actions for your robot.
 - `map()`, `position()` and `battery_soc()` each allows RMF to retrieve the relevant data from your robot. The robot state information helps RMF to allocate tasks, plan routes and initiate charging when needed.
2. For **each** robot fleet, create a `config.yaml` file to include important fleet parameters. These parameters will be passed to the fleet adapter and configured when initializing the fleet. Do note that you must list all robots in the fleet under `robots` to ensure that they are registered to the RMF fleet.
3. [Optional] Create a fleet manager that interfaces with your fleet's robots if you'd like a consolidated manager for your fleet, rather than having RMF communicate with the individual robots. This fleet manager would be a bridge between the RMF fleet adapter and your fleet robots, as depicted in the diagram above. It should be able to relay navigation commands to the fleet robots and also report their robot statuses. If you have multiple fleets using different robot APIs, make sure to create separate fleet managers for these fleets.

If you have opted to use a fleet manager, you may find it helpful to include any relevant data in the fleet `config.yaml`, demonstrated in the `fleet_manager` section [here](#).

You may refer to the [Fleet Adapter Tutorial](#) section in this Book or `rmf_demos` `RobotClientAPI` for examples and in-depth step-by-step instructions.

Once you are done, you can run both the fleet adapter and any custom fleet manager. Remember to parse the configuration file and navigation graph when launching the adapter.